

CS121 Data Structures in C

11 Hashing

Outline

- Hashing
- Hash table
- Hash function
- Collision handling
 - ◆ Separate Chaining
 - ◆ Open Addressing
 - Linear probing
 - Quadratic probing
 - Double hash
- Rehashing

Hash Table

- **Hash table** is a data structure that **supports**
 - ♦ **finds** (we just want to know whether the key is in the data structure or not).
 - ♦ **insertions** (it is for building up the data structure).
 - ♦ **deletions** (deletions may be unnecessary in some applications).
- The implementation of hash tables is called **hashing**.
 - ♦ A technique which allows efficient executions of the above operations: finds, insertions and deletions (constant average time).
- For the hash tables, operations that require any ordering information among elements are **not supported**, e.g.
 - ♦ **findMin** and **findMax**
 - ♦ Successor and predecessor
 - ♦ Report data within a given range
 - ♦ List out the data in order

Ideal Hash Table

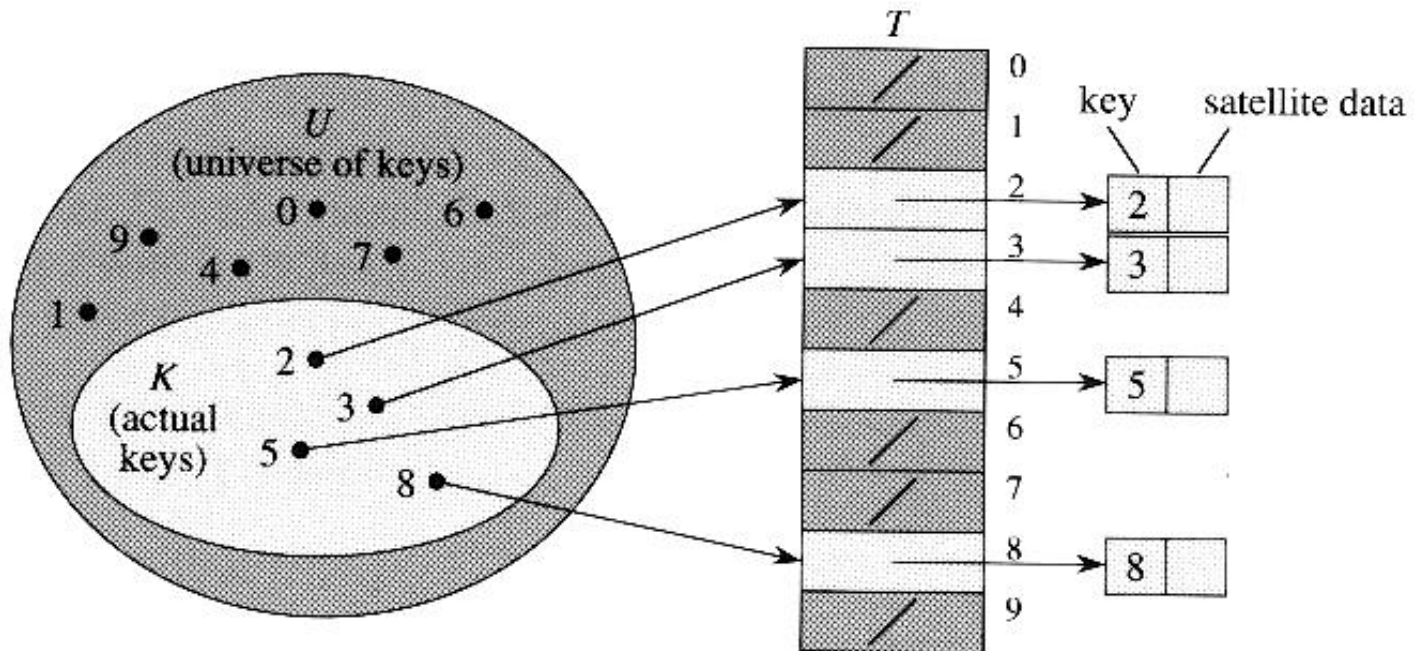
- The **ideal** hash table data structure is an array of some fixed size, containing the items.
- A search is performed based on key.
- Each key is mapped into some position in the range **0** to **TableSize-1**.
(Refer to the table size as **TableSize**.)
- The mapping is called **hash function**.

0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	

A hash table

Ideal Hash Table (Illustration)

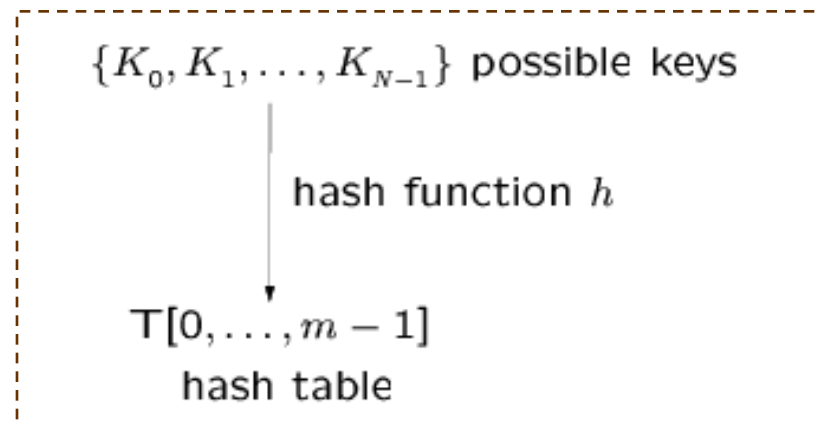
- U = universe of keys.
- In the ideal hash table, each position (slot) in the array corresponds to a key in U .
 - ♦ $T[k]$ points to an element with key k .
 - ♦ If there is no element with key k , then $T[k]=\text{NULL}$.



Ideal Hash Table

- Insertion, deletion and finds all take $O(1)$ time in the worst case.
- Problem: waste too much space if the universe is too large compared with the actual number of elements to be stored.
 - ♦ For example, student IDs are 8-digit integers, so the universe size is 10^8 (i.e., 100 million slots available). But we only have about 7000 students (we only need 0.007 million slots).

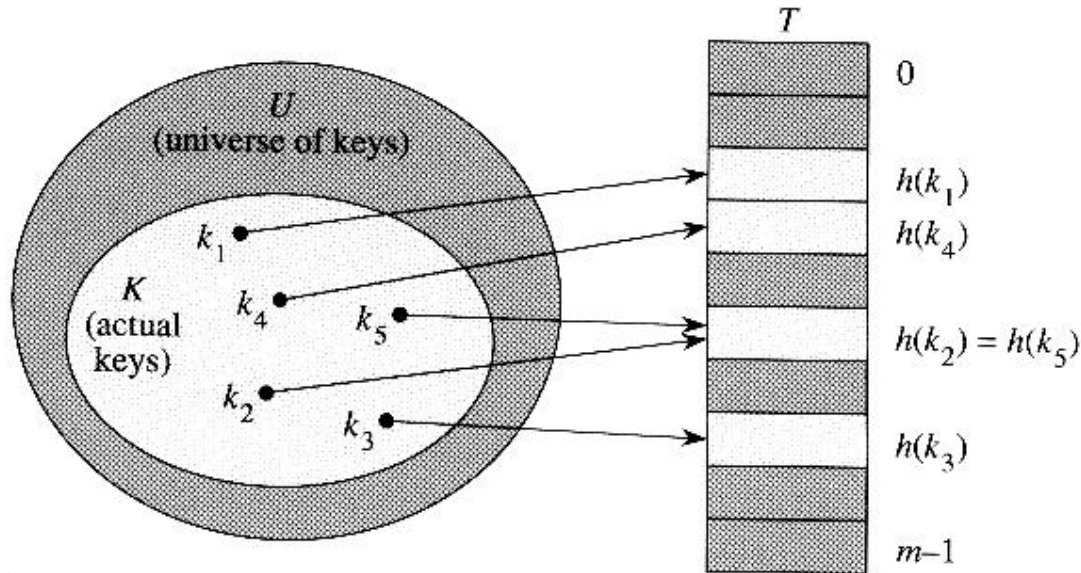
The actual
hash table



- N = size of the universe of keys; m = hash table size.
- Usually, $m \ll N$ (The table size is much smaller than the number of actual keys).

Hash Function

- With hashing, an element of key k is stored in $T[h(k)]$

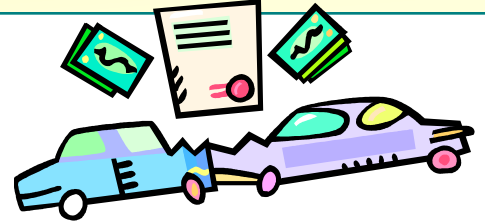


- h : hash function
 - Maps the universe of keys, U , into the slots of a *hash table* $T[0,1,\dots,m-1]$.
 - An element of key k *hashes* to the slot $h(k)$.
 - $h(k)$ is the *hash value* of key k .
 - The load factor λ , the ratio of the number of elements in the hash table to the table size.

Collision

- Problem: *collision*

- ◆ Two keys may hash to the same slot.
- ◆ Can we ensure that any two distinct keys get different slots?
 - No, $N > m$, where m is the size of the hash table.



- To deal with the problem of collision, there are two tasks. Task 1 and Task 2, as listed below.

- Task 1: Design a good hash function

- It is fast to compute; and
- It can minimize the number of collisions.

- Task 2: Design a method to resolve the collisions when they occur

Design Hash Function

The rules for choosing the hash function

- ♦ Simplicity of calculating
- ♦ Less collision and better distribution
- A simple and reasonable strategy: $h(\text{key}) = \text{key} \bmod \text{TableSize}$
 - ♦ e.g. $\text{TableSize}=12$, $\text{key}=100$, $h(k)=4$, where 4 = remainder of $100/12$.
 - ♦ Requires only a single division operation (quite fast).

Notes:

- ♦ Certain values of **TableSize** should be avoided.
 - e.g. if the keys are decimal numbers, it should not set **TableSize** to be a power of 10. It doesn't depend on all the digitals in the numbers and results in the collision. The hash function should take all the digitals into consideration.
- ♦ It's a good practice to set the **TableSize** to be a **prime number**.

Design Hash Function

- ◆ It's a good practice to set the **TableSize** to be a **prime number**.

Example 1. the hash table is to hold 2000 binary numbers.

- a. We choose TableSize=673, the next larger prime number of $2000/3=667$ (we don't mind an average of 3 numbers being hashed to the same entry) or slightly larger;
- b. The selected prime number should not be too close to the exact powers of 2 ($2^9 = 512 < 673 < 2^{10} = 1024$).

Example 2. keys: 10, 15, 20, 25, 30, ...

If we choose TableSize=15, then 10→T[10], 15→T[0], 20→T[5], 25→T[10],.....only three indexes are occupied. Collision occurs!

We choose the prime 13 as TableSize, then 10→T[10], 15→T[2], 20→T[7], 25→T[12], 30→T[4],...no collision occurs!

Collision Handling

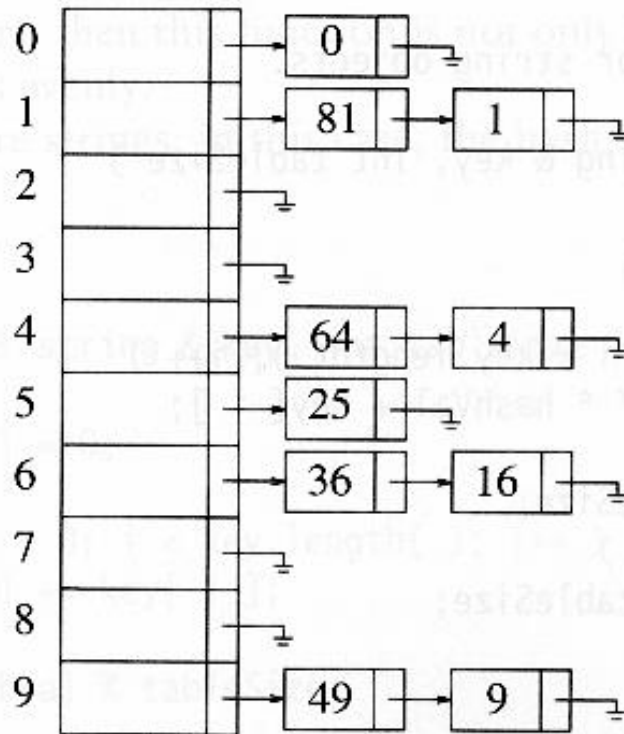
Collision Handling

- Collision: when an element is inserted, it hashes to the same value as an already inserted element.
- Solution
 - ◆ Separate Chaining
 - ◆ Open Addressing

1. Separate Chaining

- We use a table of linked lists to keep a linked list of keys that hash to the same value.

Example



Keys:

All keys are perfect squares of 0~9.

Hash function:

$$h(K) = K \bmod 10$$

(The table size is not a prime for simplicity.)

Implementation of Separate Chaining Operations

- Type declarations of the separate chaining

```
struct ListNode;
typedef struct ListNode *Position;
struct HashTbl;
typedef struct HashTbl *HashTable;

HashTable InitializeTable( int TableSize );
void DestroyTable( HashTable H );
Position Find( ElementType Key, HashTable H );
void Insert( ElementType Key, HashTable H );
ElementType Retrieve( Position P );
/* Routines such as Delete are MakeEmpty are omitted */

#define MinTableSize (10)
struct ListNode
{
    ElementType Element;
    Position     Next;
};

typedef Position List;
struct HashTbl
{
    int TableSize;
    List *TheLists;
};
```

Implementation of Separate Chaining Operations

- Initialization

```
HashTable
InitializeTable( int TableSize )
{
    HashTable H;
    int i;

    if( TableSize < MinTableSize )
    {
        Error( "Table size too small" );
        return NULL;
    }

    /* Allocate table */
    H = malloc( sizeof( struct HashTbl ) );
    if( H == NULL )
        FatalError( "Out of space!!!" );

    H->TableSize = NextPrime( TableSize );

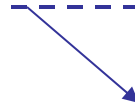
    /* Allocate array of lists */
    H->TheLists = malloc( sizeof( List ) * H->TableSize );
    if( H->TheLists == NULL )
        FatalError( "Out of space!!!" );
}
```

~ go to the next page ~

Implementation of Separate Chaining Operations

- Initialization (Cont'd)

```
.....  
  
/* Allocate list headers */  
for( i = 0; i < H->TableSize; i++ )  
{  
    H->TheLists[ i ] = malloc( sizeof( struct ListNode ) );  
    if( H->TheLists[ i ] == NULL )  
        FatalError( "Out of space!!!" );  
    else  
        [H->TheLists[ i ]->Next = NULL;]  
}  
  
return H;  
}
```



Make empty

Implementation of Separate Chaining Operations

- Find

```
Position
Find( ElementType Key, HashTable H )
{
    Position P;
    List L;
    //decide the list
    L = H->TheLists[ Hash( Key, H->TableSize ) ];
    P = L->Next;
    while( P != NULL && P->Element != Key )
        /* Probably need strcmp!! */
        P = P->Next;
    return P;
}
```

Implementation of Separate Chaining Operations

- Insertion

```
void
Insert( ElementType Key, HashTable H )
{
    Position Pos, NewCell;
    List L;

    Pos = Find( Key, H );
    if( Pos == NULL )    /* Key is not found */
    {
        NewCell = malloc( sizeof( struct ListNode ) );
        if( NewCell == NULL )
            FatalError( "Out of space!!!" );
        else
        {
            L = H->TheLists[ Hash( Key, H->TableSize ) ];
            NewCell->Next = L->Next;
            NewCell->Element = Key;  /* Probably need strcpy! */
            L->Next = NewCell;
        }
    }
}
```

Implementation of Separate Chaining Operations

- Deletion
 - ◆ A straightforward implementation of deletion in a linked list.
 - ◆ If the deletion isn't required, it is probably best to not use headers, because
 - It causes the complicated operation
 - Waste considerable space
- Other implementation for solving the collisions
 - ◆ A binary search tree or even other hash table
 - ◆ But if the table is large, the hash function is good and the lists are short, no complicated implementation is necessary

Separate Chaining Features

- The general rule for separate chaining is to make the hash table size about as large as the number of element expected.
- If the hash function works well, the number of keys in each linked list will be a **small constant**. Therefore, we expect that each search, insertion, and deletion can be done in **constant time**. (In analysis, the size of the hash table is not really important.)
- **Disadvantage**: Memory allocation in linked list manipulation will slow down the program.
- **Advantage**: deletion is easy because it is just an item removal in a linked list.

2. Open Addressing

- Open addressing: **relocate the key K to be inserted if it collides** with an existing key.
 - ♦ We store K at an entry different from $\text{Table}[h(K)]$.
- Two issues arise
 - ♦ What is the relocation scheme?
 - ♦ How to search for K later?
- Three common methods for resolving a collision in open addressing
 - ♦ Linear probing
 - ♦ Quadratic probing
 - ♦ Double hashing

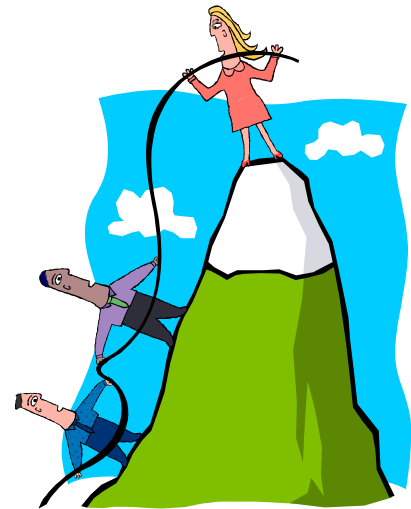
Open Addressing Strategy

- To insert a key K , compute $h_0(K)$. If $\text{Table}[h_0(K)]$ is empty, insert it there. If collision occurs, probe alternative slots $h_1(K)$, $h_2(K)$, until an empty cell is found.

$h_i(K) = (\text{hash}(K) + f(i,K)) \bmod \text{TableSize}$, with $f(0,K) = 0$

- ♦ K = input key.
- ♦ $\text{hash}(K)$ gives the home position or the primary slot.
- ♦ f = collision resolution strategy.
 - It gives the probe sequence:
 - $\text{hash}(K)+f(0,K)$, $\text{hash}(K)+f(1,K)$, $\text{hash}(K)+f(2,K)$,
- ♦ The probe pattern is
 - $f(0,K)$, $f(1,K)$, $f(2,K)$,
- ♦ i = probe number, $i = 0, 1, 2, \dots$

All the data go inside the table, a bigger table is needed for open addressing hashing than for separate chaining hashing.



Open Addressing Strategy

- The collision resolution strategy
 - ♦ Linear probing
 - ♦ Quadratic probing
 - ♦ Double hashing

Strategy One: Linear Probing

- The collision resolution strategy
 - ♦ **f is a linear function of i**
i is the probe number, $i=0,1,2,\dots$
typically, $f(i,K) = i$
 - Cells in the hash table are probed **sequentially** (with wraparound)
 - ♦ **Hashing function: $h_i(K) = (\text{hash}(K) + i) \bmod \text{TableSize}$**
- Insertion
 - ♦ Let K be the new key to be inserted, compute $\text{hash}(K)$
 - ♦ For $i = 0$ to $\text{TableSize}-1$
 - Compute $L = (\text{hash}(K) + i) \bmod \text{TableSize}$
 - $T[L]$ is empty, then we put K there and stop.
 - ♦ If we cannot find an empty entry to put K, it means that the table is full and we should report an error.

Strategy Two: Quadratic Probing

- $f(i,K) = i^2$ $h_i(K) = (\text{hash}(K) + i^2) \bmod \text{TableSize}$
- The quadratic probe pattern, 1, 4, 9, 16, ... , help avoid the growth of clusters.
- The probe patterns are the same for keys with different home positions, although the probe sequences are different.
 - ♦ e.g. For TableSize=101, $\text{hash}(k_1)=30$, $\text{hash}(k_2)=29$
 - ♦ The probe sequence for k_1 : 30, 31=30+1, 34=30+4, 39=30+9, ...
 - ♦ The probe sequence for k_2 : 29, 30=29+1, 33=29+4, 38=29+9, ...
- About deletion
 - ♦ Standard deletion can't be performed in an open addressing hash table, because the cell might have caused a collision to go past it. Remove it, all of the remaining **Finds** will fail.
 - ♦ Using lazy deletion
 - Deletions are done by marking an element as deleted, rather than erasing it entirely.

Implementation of Open Addressing Quadratic Probing Hash Table

- Interface for quadratic probing hash table

```
typedef unsigned int Index;
typedef Index Position;

struct HashTbl;
typedef struct HashTbl *HashTable;

HashTable InitializeTable( int TableSize );
void DestroyTable( HashTable H );
Position Find( ElementType Key, HashTable H );
void Insert( ElementType Key, HashTable H );
ElementType Retrieve( Position P, HashTable H );
HashTable Rehash( HashTable H );
/* Routines such as Delete are MakeEmpty are omitted */

struct HashEntry
{
    ElementType      Element;
    enum KindOfEntry Info;
};

typedef struct HashEntry Cell;

/* Cell *TheCells will be an array of */
/* HashEntry cells, allocated later */
struct HashTbl
{
    int TableSize;
    Cell *TheCells;
};
```

Implementation of Open Addressing Quadratic Probing Hash Table

- Initialization

```
HashTable
InitializeTable( int TableSize )
{
    HashTable H;
    int i;

    if( TableSize < MinTableSize )
    {
        Error( "Table size too small" );
        return NULL;
    }

    /* Allocate table */
    H = malloc( sizeof( struct HashTbl ) );
    if( H == NULL )
        FatalError( "Out of space!!!" );

    H->TableSize = NextPrime( TableSize );

    /* Allocate array of Cells */
    H->TheCells = malloc( sizeof( Cell ) * H->TableSize );
    if( H->TheCells == NULL )
        FatalError( "Out of space!!!" );

    for( i = 0; i < H->TableSize; i++ )
        H->TheCells[ i ].Info = Empty;

    return H;
}
```

Implementation of Open Addressing Quadratic Probing Hash Table

- Find(Key, H)

```
Position
Find( ElementType Key, HashTable H )
{
    Position CurrentPos;
    int CollisionNum;

    CollisionNum = 0;
    CurrentPos = Hash( Key, H->TableSize );
    while( H->TheCells[ CurrentPos ].Info != Empty &&
           H->TheCells[ CurrentPos ].Element != Key )
        /* Probably need strcmp!! */
    {
        CurrentPos += 2 * ++CollisionNum - 1;
        if( CurrentPos >= H->TableSize )
            CurrentPos -= H->TableSize;
    }
    return CurrentPos;
}
```

Implementation of Open Addressing Quadratic Probing Hash Table

- Insert(Key, H)

```
void
Insert( ElementType Key, HashTable H )
{
    Position Pos;

    Pos = Find( Key, H );
    if( H->TheCells[ Pos ].Info != Legitimate )
    {
        /* OK to insert here */
        H->TheCells[ Pos ].Info = Legitimate;
        H->TheCells[ Pos ].Element = Key;
        /* Probably need strcpy! */
    }
}
```

Strategy Two: Quadratic Probing

- *Secondary clustering*

- ♦ Quadratic probing eliminates primary clustering, but it still has the secondary clustering
- ♦ Keys that hash to the same home position will probe the same alternative slots (same probe patterns)
 - 1, 4, 9,
 - Because the $f(i, K) = i^2$ is the same for those keys and independent on the key K .
 - The search length can still be increased.
- ♦ To avoid secondary clustering, the probe sequence needs to be a function of K (the original key value).

Strategy Three: Double Hashing

- To alleviate the problems of primary clustering and secondary clustering, two hash functions: **hash (K)** and **hash2 (K)**, are used.
- **$f(i, K) = i * \text{hash2}(K)$**
 - ♦ f (the probe pattern) now depends on the probe number i and the input K .
 - ♦ Poor choice of **hash2 (K)** would be disastrous.
 - ♦ **hash2 (K)** must **avoid zero** return value. Otherwise, **$f(i, K) = 0$** when i is not **zero**.
 - ♦ **hash2 (K)** should depend on K .
 - If **hash (K) = 0** and **hash2 (K) = TableSize/2** (half the table size), then we can only examine the entries **$T[0]$** , **$T[\text{TableSize}/2]$** , and nothing else! In this case, we only be able to examine a fraction of the table entries.
 - E.g., **$\text{hash2}(K) = R - (K \bmod R)$** , with **$R$** is a prime number smaller than **TableSize**, will work well.

Rehashing

- If the table gets too full, the running time for the operations will start taking too long and Inserts might fail for open addressing with quadratic resolution.
 - ♦ Build another table that is about twice as big (with an associated new hash function)
 - ♦ Move the entire original hash table
 - Computing the new hash value for each (nondeleted) element and inserting it in the new table

Example:

Hash function is $h(X) = X \bmod 7$.

Collision strategy: linear probing

New hash function is $h(X) = X \bmod 17$.

The rehashing is a very expensive operation. The running time is $O(N)$.

Open addressing hash table with linear probing with input 13, 15, 6, 24

0	6
1	15
2	23
3	24
4	
5	
6	13

Over 70% full
----->

0	
1	
2	
3	
4	
5	
6	6
	23
	24
	13
15	15
16	

Open addressing hash table with linear probing after 23 is inserted

Open addressing hash table after rehashing

Implementation of Rehashing

```
HashTable
Rehash( HashTable H )
{
    int i, OldSize;
    Cell *OldCells;

    OldCells = H->TheCells;
    OldSize  = H->TableSize;

    /* Get a new, empty table */
    H = InitializeTable( 2 * OldSize );

    /* Scan through old table, reinserting into new */
    for( i = 0; i < OldSize; i++ )
        if( OldCells[ i ].Info == Legitimate )
            Insert( OldCells[ i ].Element, H );

    free( OldCells );

    return H;
}
```

Summary

- Hashing
 - ♦ Hash table is a data structure that supports
 - finds
 - insertions
 - deletions
 - ♦ The implementation of hash tables is called hashing.
- Hash function
 - ♦ Maps the keys into the slots of a hash table
- Collision
 - ♦ Two keys may hash to the same slot.
 - ♦ Collision handling
 - Separate Chaining: linked list
 - Open Addressing
 - Linear probing
 - Quadratic probing
 - Double hash
- Rehashing

Assignment 12

- Written. Textbook Exercise 5.1, 5.2

Hint for 5.2: the new table size is roughly twice as large as the old one and it must be a prime. (Choose the new table size < 20)

Submit it to Moodle